CyberSecIntel
P1.2 Delivery Report
Basic Ingestion and Landing

Dídac Cayuela & Sindri Másson

March 2026

1 Context and Objective

This document summarizes the implementation delivered for P1.2 (Basic Ingestion and Landing) of the CyberSecIntel project. It is aligned with:

- `Project Statement - P1.pdf`
- `Project Statement - Context.pdf`
- previous team submission `BDM_Project_P1_1.pdf`

Main objective for this increment:

- automate ingestion from multiple cybersecurity sources,
- land raw data with traceable metadata in object storage,
- ensure the unstructured branch is also automatically ingested,
- prepare a scalable base for trusted and exploitation layers in P2.

2 Design Revision from P1.1

The original P1.1 proposal centered the offline branch on CIC-IDS2017 CSV and PCAP downloads. During P1.2 implementation, that idea proved weaker for the automation objective because the unstructured branch depended on manual dataset staging before the pipeline could ingest anything.

For this reason, the original P1.1 idea was revised during implementation after identifying a more robust fully automatic ingestion path for unstructured PCAP data. Instead of relying on manually staged CIC files, the P1.2 implementation now discovers and downloads CTU scenario artifacts automatically from the public CTU hosting server.

This revision preserves the original architectural intent:

- unstructured PCAP data is still stored in native format,
- the artifacts remain suitable for future replay into the hot path,
- the dataset branch still supports the project's big-data narrative.

At the same time, it improves compliance with the project statement's preference for automated ingestion from non-local sources.

3 How Ingestion Into Landing Is Organized

The ingestion layer is organized around two Airflow DAGs with clear responsibilities:

- **`cybersecintel_dataset_artifact_ingestion`**: automatic remote dataset-artifact DAG for CTU PCAP discovery and download from the public dataset server.
- **`cybersecintel_api_expansion_ingestion`**: public API/feed DAG for KEV, NVD, URLhaus, EPSS, CIRCL, optional ThreatFox, optional Shodan, plus synthetic stream placeholder ingestion.

Landing writes are implemented through a storage abstraction with backend set to MinIO (`LANDING_BACKEND=minio`). Data is stored directly in `s3://landing` (no local landing duplication).

Operational rules currently implemented:

- partitioning by `ingest_date=YYYY-MM-DD` (and `hour=HH` for stream),
- stable object naming per source and scenario partition,
- metadata sidecars with origin, checksum, size, and selection context,
- per-source manifest maintenance for traceability and reruns.

4 Data Sources: Origin, Type, and Role

For clarity, the main acronyms used in this section are expanded here: CISA = Cybersecurity and Infrastructure Security Agency; KEV = Known Exploited Vulnerabilities; NVD = National Vulnerability Database; CVE = Common Vulnerabilities and Exposures; FIRST = Forum of Incident Response and Security Teams; EPSS = Exploit Prediction Scoring System; CIRCL = Computer Incident Response Center Luxembourg; IOC = indicator of compromise; CTU = Czech Technical University; and PCAP = packet capture.

The selected source mix intentionally combines structured vulnerability intelligence, semi-structured enrichment and IOC feeds, and unstructured packet captures. CISA KEV, NVD, and FIRST EPSS provide stable identifiers and prioritization signals that are immediately useful in the cold path and later suitable for warm aggregates and ranking models. URLhaus, CIRCL, ThreatFox, and optionally Shodan add semi-structured context about malicious infrastructure, vulnerability details, and external exposure, which is more useful once correlated in the warm path. CTU-13 contributes raw packet evidence to the cold path and provides realistic material for future hot-path replay, while the current hot path remains represented by the synthetic IDS event producer delivered in P1.2.

Source and origin	Type	How we ingest it	What it provides
CISA KEV from the CISA Known Exploited Vulnerabilities feed	Structured	Scheduled API pull to structured landing prefix	Canonical list of already exploited CVEs, used as a strong cold-path prioritization baseline and later for warm-path ranking logic.
NVD CVE v2 from the National Vulnerability Database API	Semi-structured	Incremental API pull with pagination controls	Detailed CVE metadata, descriptions, scores, configurations, and references that support enrichment, filtering, and later normalization.
URLhaus recent from the URLhaus public CSV feed	Semi-structured	Scheduled feed/API pull to semi-structured prefix	IOC context about malicious URLs and related infrastructure, useful as enrichment material and future warm-path correlation input.
FIRST EPSS from the FIRST Exploit Prediction Scoring System API	Structured	Paginated API pull to structured prefix	Stable exploit-likelihood scores for CVEs, used to prioritize which vulnerabilities deserve deeper enrichment and later aggregation.
CIRCL Vulnerability-Lookup from the CIRCL vulnerability API	Semi-structured	API calls for top EPSS CVEs in current partition	Additional CVE-level enrichment, adding related vulnerability context beyond the core NVD records.
abuse.ch ThreatFox from the ThreatFox API	Semi-structured	Optional API pull with auth key; skips cleanly when key is absent	IOC feed with malicious indicators that enrich the cold path and can also seed later infrastructure lookups.
Shodan Host from the seeded Shodan API workflow	Semi-structured	Optional host lookups when explicitly enabled	Targeted exposure details for selected external IP indicators, used as optional enrichment rather than a primary discovery source.
CTU-13 remote PCAP artifacts from the public CTU dataset host	Unstructured	Automatic remote discovery and profile-based download from scenarios `42..54`	Raw packet evidence for the cold path and realistic artifacts for future replay into the hot path.

5 Official Source References

The implementation uses the following official pages and API/feed endpoints as source references:

- CISA KEV catalog page
- CISA KEV JSON feed endpoint

- NVD developer documentation
- NVD CVE API v2 endpoint
- URLhaus recent CSV feed
- FIRST EPSS API documentation
- CIRCL Vulnerability-Lookup documentation
- ThreatFox API documentation
- Shodan developer API documentation
- CTU-13 reference page
- CTU scenario artifact host

6 Automatic CTU Artifact Discovery

The unstructured branch is referenced through the official Stratosphere IPS CTU-13 page, but the actual machine-accessible artifacts are hosted on the public CVUT dataset server.

Reference page:

- `https://www.stratosphereips.org/datasets-ctu13`

Artifact host:

- `https://mcfp.felk.cvut.cz/publicDatasets/`

Discovery strategy:

- inspect the public directory index,
- match CTU botnet scenario folders numbered `42` through `54`,
- inspect the files available in each scenario directory,
- select a preferred capture artifact according to the active profile.

Profiles:

- `subset`: default for development and demos; processes only the first successful scenario and prioritizes the botnet-only `.pcap` capture when available.
- `full`: heavier validation path; processes all scenarios `42..54` and selects one artifact per scenario, preferring truncated `.pcap.bz2`, then other compressed `.pcap.bz2`, and finally plain `.pcap`.

7 Landing Zone Structure and Metadata

Main prefixes currently in use:

- `structured/kev/`, `structured/epss/`
- `semi_structured/nvd/`, `semi_structured/urlhaus/`
- `semi_structured/circl_vulnlookup/`

- semi_structured/threatfox/, semi_structured/shodan_seeded/
- unstructured/pcap/source=ctu13/scenario=<n>/
- stream/ids_alerts/
- metadata/manifests/

Each ingested artifact is accompanied by:

- raw source-native payload,
- *.meta.json with source URL, scenario context, retrieval timestamp, checksum, size, and selection rule,
- manifest entry (metadata/manifests/ingest_date=.../<source>.jsonl) for auditability.

8 Hot, Warm, and Cold Paths

Cold path (implemented):

- persistent raw source data in MinIO landing,
- partitioned by ingest date for reproducible backfills.

Hot path (partially implemented):

- stream branch represented by stream/ids_alerts/ ingestion,
- current source is a synthetic IDS-event placeholder while keeping the path ready for future replay-based producers.

Warm path (planned scaffold present):

- warm/stream_aggregates/ prefix exists in landing setup,
- aggregation/materialization logic is reserved for the next iteration.

9 Implemented Technologies and Components

- **Orchestration**: Apache Airflow DAGs for public-source ingestion and remote dataset artifacts.
- **Object storage**: MinIO as landing zone (s3://landing).
- **Messaging infrastructure**: Kafka + Zookeeper, currently used for the stream placeholder.
- **Metadata/state**: Postgres for Airflow metadata; JSON sidecars + manifests for lineage.
- **Runtime**: Docker Compose stack for local reproducible execution.
- **Ingestion code**: Python source-specific adapters under ingestion/batch/.
- **Dataset branch**: CTU discovery/downloader under ingestion/datasets/; stream scaffold under ingestion/stream/.

10 Current Implementation Diagram

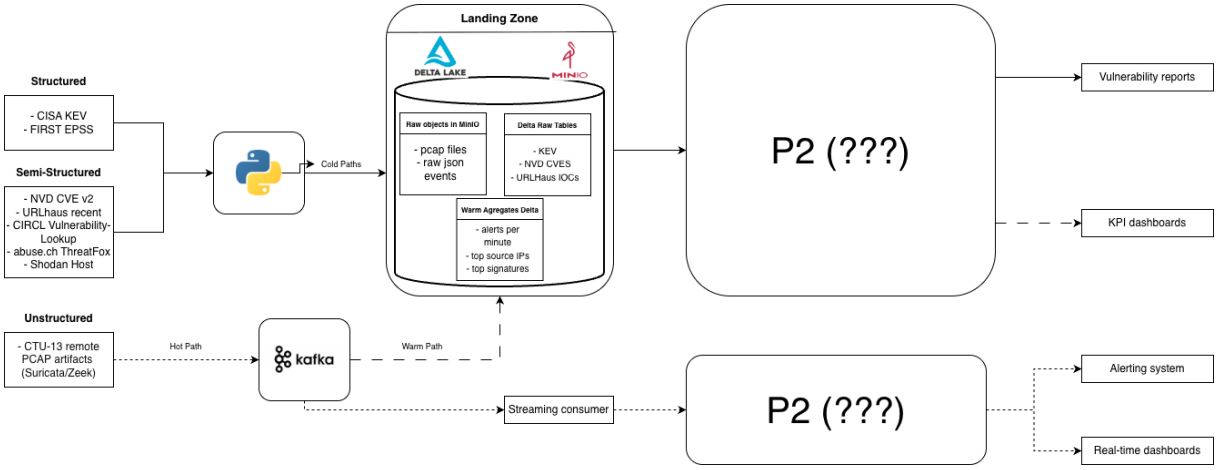

Figure 1: Current implementation architecture for the P1.2 ingestion and landing pipeline.

11 Current Limitations and Next Iteration

- Remote dataset links may change over time, so the CTU discovery rules and URLs should remain easy to update.
- The `full` CTU profile is heavier in bandwidth, runtime, and storage than the default `subset` profile.
- ThreatFox and Shodan depend on API-key availability; Shodan is disabled by default.
- The hot path currently uses a synthetic placeholder; next step is replay-based event production from landed artifacts.
- Next iteration: trusted normalization (Silver), warm aggregates, and exploitation layer materialization.